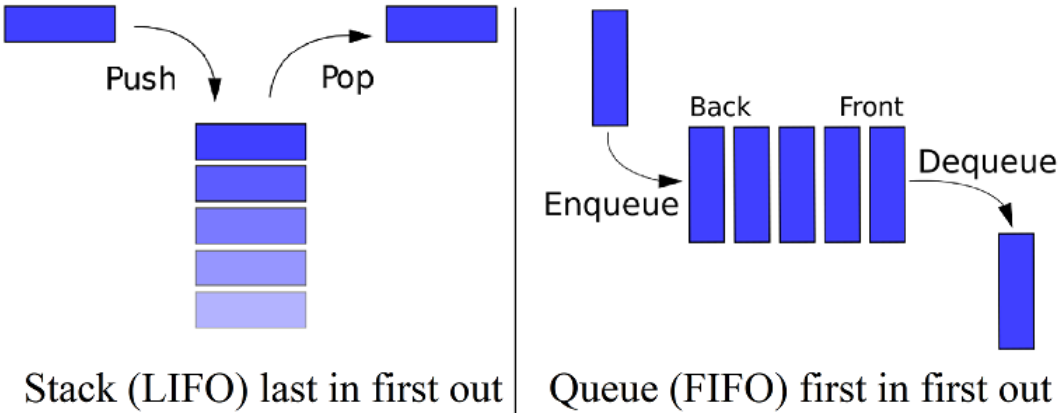
Stack and Queue

Debarka Sengupta, Ph.D., FNASc.

Content

- Stack and Queue Visualization
  - **Stack (LIFO - Last In, First Out):**
  - **Queue (FIFO - First In, First Out):**
- Array Implementation
  - **Functions**
  - **C++ pseudocode**
- Mathematical Notation Translation
  - **Operand and operator**
  - **Precedence**
  - **Mathematical notation translation**
  - **Infix to postfix**
  - **Example**
- Linked list implementation of stack.
- Linked Queue
  - What is a linked queue?
  - Advantage over array-based implementation.
  - Implementation
  - A few concepts
- Circular queue

Stack and Queue Visualization



Stack/Queue depiction.

Stack (LIFO - Last In, First Out):

- A stack is a data structure that operates on the principle of Last In, First Out (LIFO).
- Imagine a stack of plates: you can only add or remove plates from the top.
- In a stack, elements are inserted (pushed) onto the top of the stack and removed (popped) from the top as well.
- The last element pushed onto the stack is the first one to be popped out.
- Stacks are commonly used in programming for tasks like managing function calls (the call stack), undo mechanisms, and parsing expressions.

Queue (FIFO - First In, First Out):

- A queue is a data structure that operates on the principle of First In, First Out (FIFO).
- Think of a queue as a line of people waiting for something: the person who arrives first is served first.
- In a queue, elements are inserted (enqueued) at the rear end and removed (dequeued) from the front end.
- The first element enqueued into the queue is the first one to be dequeued.
- Queues are used in scenarios such as task scheduling, breadth-first search algorithms, and message passing systems.

Array Implementation

Functions

1. **CreateStack(Stack &s)**:
   - Task: Initializes the stack.
   - Brief Description: Sets the top index of the stack to -1, indicating an empty stack.
2. **Push(Stack &s, int value)**:
   - Task: Adds an element to the top of the stack.
   - Brief Description: Inserts the given value onto the top of the stack if there is space available.
3. **Pop(Stack &s)**:
   - Task: Removes and returns the element at the top of the stack.
   - Brief Description: Removes and returns the top element from the stack if the stack is not empty.
4. **Peek(Stack &s)**:
   - Task: Returns the element at the top of the stack without removing it.

- Brief Description: Returns the top element of the stack without modifying the stack itself.

5. **IsFull(Stack &s)**:
   - Task: Checks if the stack is full.
   - Brief Description: Returns true if the stack is full and false otherwise.
6. **IsEmpty(Stack &s)**:
   - Task: Checks if the stack is empty.
   - Brief Description: Returns true if the stack is empty and false otherwise.

C++ pseudocode

```
#include <iostream>

#define MAX_SIZE 100

using namespace std;

// Define the structure for the stack
struct Stack {
    int items[MAX_SIZE];
    int top;
};

// Function to create an empty stack
void CreateStack(Stack &s) {
    s.top = -1; // Initialize top to -1 indicating an empty stack
}

// Function to check if the stack is full
bool IsFull(Stack &s) {
    return s.top == MAX_SIZE - 1; // Stack is full if top is equal to MAX_SIZE - 1
}

// Function to check if the stack is empty
bool IsEmpty(Stack &s) {
    return s.top == -1; // Stack is empty if top is equal to -1
}

// Function to push an element onto the stack
void Push(Stack &s, int value) {
    if (IsFull(s)) {
        cout << "Stack Overflow: Cannot push element onto full stack\n";
        return;
    }
    s.items[++s.top] = value; // Increment top and insert the value into the stack
}

// Function to pop an element from the stack
```

```
int Pop(Stack &s) {
    if (IsEmpty(s)) {
        cout << "Stack Underflow: Cannot pop element from empty stack\n";
        return -1;
    }
    return s.items[s.top--]; // Decrement top and return the popped element
}

// Function to return the top element of the stack without removing it
int Peek(Stack &s) {
    if (IsEmpty(s)) {
        cout << "Stack is empty\n";
        return -1;
    }
    return s.items[s.top]; // Return the top element of the stack
}

int main() {
    Stack myStack;
    CreateStack(myStack);

    // Push elements onto the stack
    Push(myStack, 10);
    Push(myStack, 20);
    Push(myStack, 30);

    // Peek at the top element
    cout << "Top element: " << Peek(myStack) << endl;

    // Pop elements from the stack
    cout << "Popped element: " << Pop(myStack) << endl;
    cout << "Popped element: " << Pop(myStack) << endl;

    // Peek again
    cout << "Top element: " << Peek(myStack) << endl;

    return 0;
}
```

💡 Time complexity of each of the above functions: **O (1).**

💡 **Why `Stack &s`?**

In the function declarations and definitions provided, `Stack &s` signifies that the parameter `s` is passed by reference. Let's break it down:

- `Stack`: This is the name of the struct type.
- `&`: This is the reference operator in C++, indicating that `s` is a reference to an object of type `Stack`.
- `s`: This is the name of the parameter.

Passing the `Stack` object by reference (`&`) allows the function to directly modify the original `Stack` object passed to it, rather than creating a copy of the object. This is more memory-efficient, especially when dealing with large data structures, as it avoids unnecessary copying of data.

In summary, `Stack &s` in these function declarations and definitions means that the functions operate on the `Stack` object directly, using a reference to it. Any modifications made to `s` within the function will affect the original `Stack` object passed to it.

Problems in passing by value.

If you simply passed `Stack s` instead of `Stack &s` as a parameter to the functions, it would result in passing the `Stack` object by value instead of by reference.

Here's what would happen:

1. **Passing by Value**: When you pass an object by value, a copy of the object is made and passed to the function. This means that any modifications made to the parameter `s` within the function would only affect the local copy of the `Stack` object, not the original `Stack` object passed to the function.
2. **Changes are Not Reflected Outside the Function**: Any modifications made to `s` within the function would be applied to the local copy of the `Stack` object and would not be reflected in the original `Stack` object passed from the caller. Therefore, after the function call returns, the original `Stack` object remains unchanged.
3. **Memory Overhead**: Passing large objects by value can lead to increased memory usage and decreased performance, especially for complex data structures.

In contrast, passing by reference (`Stack &s`) allows the function to directly modify the original `Stack` object passed to it, avoiding the need for copying and ensuring that any changes made within the function are reflected in the original object. This is typically preferred for efficiency and to ensure that the function has access to the original object's state.

💡 **pre/postfix increment/decrement operators**

1. **Prefix Increment (`++variable`):**
   - The prefix increment operator (`++variable`) first increases the value of the variable by 1, and then returns the updated value.
   - Example: `++x;`
   - In this case, `x` is incremented by 1, and the new value of `x` is returned.
2. **Postfix Increment (`variable++`):**
   - The postfix increment operator (`variable++`) first returns the current value of the variable, and then increases the value by 1.
   - Example: `x++;`
   - In this case, the current value of `x` is returned, and then `x` is incremented by 1.
3. **Prefix Decrement (`-variable`):**
   - The prefix decrement operator (`-variable`) first decreases the value of the variable by 1, and then returns the updated value.
   - Example: `-x;`
   - In this case, `x` is decremented by 1, and the new value of `x` is returned.
4. **Postfix Decrement (`variable--`):**
   - The postfix decrement operator (`variable--`) first returns the current value of the variable, and then decreases the value by 1.
   - Example: `x--;`
   - In this case, the current value of `x` is returned, and then `x` is decremented by 1.

Mathematical Notation Translation

Operand and operator

Operand and operator

Precedence

Symbol Used	Operation Performed	Precedence
* (asterisk)	Multiplication	Highest
/ (slash)	Division	Highest
% (percentage)	Modulus (Remainder)	Highest
+ (plus)	Addition	Lowest
– (hyphen)	Subtraction	Lowest

Taken from RS Salaria's book.

Mathematical notation translation

Infix	A+B
Prefix	+AB
Postfix	AB+

Infix to postfix

The infix to postfix conversion is crucial for enabling computers to interpret human-readable mathematical expressions accurately. Computers lack the ability to recognize the order of operations inherent in infix expressions, whereas postfix expressions eliminate the need for parentheses and allow straightforward parsing and execution.

The postfix expression format places operators after their operands, making it easier to determine the correct sequence of calculations. This format ensures that the order of operations remains consistent regardless of the presence of nested parentheses in the original infix expression.

Applications of infix to postfix conversions include:

1. Enabling computers to efficiently calculate complex mathematical expressions.
2. Simplifying the design of compilers and interpreters.
3. Facilitating the construction of reverse Polish calculator devices.
4. Supporting the development of language processing systems.

💡 **Algorithm outline**

1. Initialize two data structures: an empty `stack` and an empty `postfix_expression_string`.
2. Scan the input infix expression character by character from left to right.
3. For each character: a) If the character is an operand (digit or variable), append it directly to the `postfix_expression_string`. b) If the character is an opening parenthesis ('('), push it onto the `stack`. c) If the character is a closing parenthesis (')'), while the stack is not empty and the top element is an opening parenthesis, repeatedly pop operators from the stack and append them to the `postfix_expression_string`, then discard the opening parenthesis. d) If the character is an operator (+, -, *, /, ...): i) While the stack is not empty and the precedence of the operator at the top of the stack is greater than or equal to the precedence of the current operator, pop operators from the stack and append them to the `postfix_expression_string`. ii) Push the current operator onto the `stack`.
4. After processing all characters in the infix expression, pop any remaining operators from the stack and append them to the `postfix_expression_string`.
5. The resulting `postfix_expression_string` contains the converted postfix expression.

Example

Example 1: Here's the step-by-step conversion of the infix expression `(A+B)*(C-D)` to postfix:

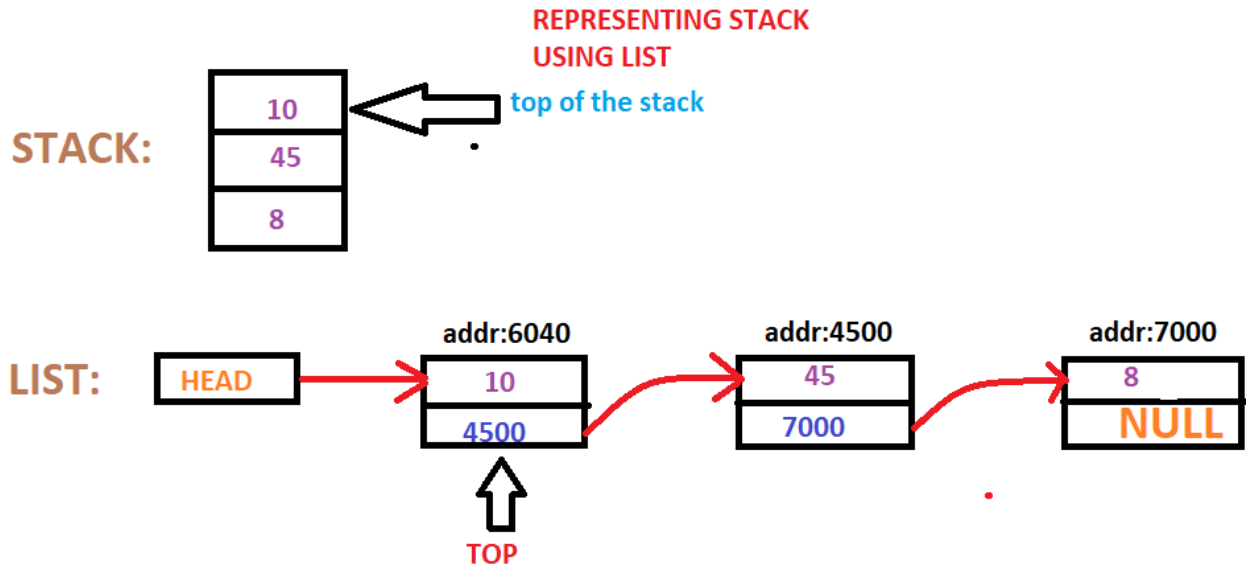
Character	Postfix Expression	Stack
(		(
A	A	(
+	A	(+
B	AB	(+
)	AB+	
*	AB+	*
(	AB+	* (
C	AB+C	* (
-	AB+C	* (-
D	AB+CD	* (-
)	AB+CD-	*
	AB+CD-*	

The resulting postfix expression is `AB+CD-*`.

Example 2: Let's apply the algorithm to convert the infix expression `(A+B)/(C-D) - (E*F)` to postfix:

Answer hint: `AB+CD-/EF*-`.

Linked list implementation of stack.




```
#include <iostream>

// Define a node for the linked list
struct Node {
    int data;
    Node* next;

    Node(int val) : data(val), next(nullptr) {} // Constructor to initialize data and nex
t pointer
};

// Define the LinkedStack class
class LinkedStack {
private:
    Node* top; // Pointer to the top of the stack

public:
    LinkedStack() : top(nullptr) {} // Constructor to initialize top pointer

    // Function to check if the stack is empty
    bool isEmpty() {
        return top == nullptr; // If top is null, the stack is empty
    }

    // Function to push an element onto the stack
    void push(int val) {
        Node* newNode = new Node(val); // Create a new node
        newNode->next = top;           // Link the new node to the current top node
        top = newNode;                 // Update top to point to the new node
        std::cout << val << " pushed onto the stack." << std::endl; // Print a message in
dicating successful push
    }
```

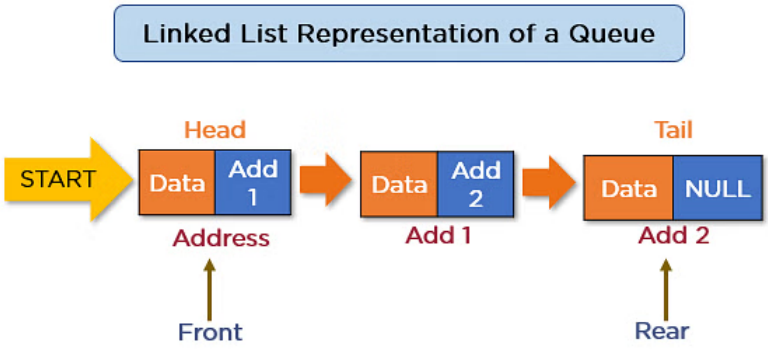
```
    // Function to pop an element from the stack
    void pop() {
        if (isEmpty()) {
            std::cout << "Stack is empty. Cannot pop." << std::endl; // If the stack is e
mpty, print an error message
            return;
        }
        Node* temp = top; // Store the current top node in a temporary variable
        top = top->next;  // Move top to the next node
        std::cout << temp->data << " popped from the stack." << std::endl; // Print a mes
sage indicating successful pop
        delete temp;      // Delete the popped node
    }

    // Function to display the stack
    void display() {
        if (isEmpty()) {
            std::cout << "Stack is empty." << std::endl; // If the stack is empty, print
a message indicating so
            return;
        }
        Node* current = top; // Start from the top of the stack
        std::cout << "Stack: ";
        while (current != nullptr) {
            std::cout << current->data << " "; // Print the data of each node
            current = current->next;           // Move to the next node
        }
        std::cout << std::endl;
    }
};

int main() {
    LinkedStack stack;      // Create a stack object
    stack.push(10);         // Push elements onto the stack
    stack.push(20);
    stack.push(30);
    stack.display();        // Display the stack
    stack.pop();            // Pop an element from the stack
    stack.display();        // Display the stack after pop
    return 0;
}
```

```
Output:
10 pushed onto the stack.
20 pushed onto the stack.
30 pushed onto the stack.
Stack: 30 20 10
```

```
30 popped from the stack.
Stack: 20 10
```

Linked Queue

What is a linked queue?



A linked queue is a data structure that represents a queue using a linked list. In a linked queue, elements are stored in a sequence where each element (node) contains a value and a pointer to the next element in the sequence.

Here are the main characteristics of a linked queue:

1. **Dynamic Size:** Unlike arrays, linked queues can dynamically adjust their size as elements are added or removed.
2. **FIFO (First-In, First-Out) Principle:** Similar to traditional queues, elements are inserted at the rear (end) of the queue and removed from the front. This ensures that the element that has been in the queue the longest is the next to be removed.
3. **Nodes:** Each element in the linked queue is represented by a node, which contains two parts: the data (value) and a pointer to the next node in the sequence.
4. **Front and Rear Pointers:** The linked queue maintains pointers to the front and rear nodes of the queue. These pointers are updated as elements are added to or removed from the queue.
5. **Operations:** Linked queues typically support operations such as enqueue (adding an element to the rear of the queue), dequeue (removing an element from the front of the queue), isEmpty (checking if the queue is empty), and display (displaying the elements of the queue).

Advantage over array-based implementation.

Linked queues are advantageous because they offer efficient insertion and deletion operations compared to array-based queues, especially when the size of the queue is not known in advance or needs to change frequently. However, they may require more memory due to the overhead of storing pointers for each node. Overall, linked queues are a fundamental data structure used in many applications, including operating systems, network protocols, and simulations.

Implementation

```
#include <iostream>

// Define a node for the linked list
struct Node {
```

```
    int data;
    Node* next;

    Node(int val) : data(val), next(nullptr) {} // Constructor to initialize data and nex
t pointer
};

// Define the LinkedQueue class
class LinkedQueue {
private:
    Node* front; // Pointer to the front of the queue
    Node* rear;  // Pointer to the rear of the queue

public:
    LinkedQueue() : front(nullptr), rear(nullptr) {} // Constructor to initialize front a
nd rear pointers

    // Function to check if the queue is empty
    bool isEmpty() {
        return front == nullptr; // If front is null, the queue is empty
    }

    // Function to enqueue an element
    void enqueue(int val) {
        Node* newNode = new Node(val); // Create a new node
        if (isEmpty()) {
            front = rear = newNode; // If the queue is empty, set both front and rear to
the new node
        } else {
            rear->next = newNode; // Otherwise, link the new node to the rear
            rear = newNode;       // Update rear to point to the new node
        }
        std::cout << val << " enqueued to the queue." << std::endl; // Print a message in
dicating successful enqueue
    }

    // Function to dequeue an element
    void dequeue() {
        if (isEmpty()) {
            std::cout << "Queue is empty. Cannot dequeue." << std::endl; // If the queue
is empty, print an error message
            return;
        }
        Node* temp = front;     // Store the front node in a temporary variable
        front = front->next;    // Move front to the next node
        if (front == nullptr) {
            rear = nullptr;     // If front becomes null, make rear null too
        }
        std::cout << temp->data << " dequeued from the queue." << std::endl; // Print a m
essage indicating successful dequeue
        delete temp;            // Delete the dequeued node
```

```
    }

    // Function to display the queue
    void display() {
        if (isEmpty()) {
            std::cout << "Queue is empty." << std::endl; // If the queue is empty, print
a message indicating so
            return;
        }
        Node* current = front; // Start from the front of the queue
        std::cout << "Queue: ";
        while (current != nullptr) {
            std::cout << current->data << " "; // Print the data of each node
            current = current->next;          // Move to the next node
        }
        std::cout << std::endl;
    }
};

int main() {
    LinkedQueue queue;       // Create a queue object
    queue.enqueue(10);       // Enqueue elements
    queue.enqueue(20);
    queue.enqueue(30);
    queue.display();         // Display the queue
    queue.dequeue();         // Dequeue an element
    queue.display();         // Display the queue after dequeue
    return 0;
}
```

```
Output:
10 enqueued to the queue.
20 enqueued to the queue.
30 enqueued to the queue.
Queue: 10 20 30
10 dequeued from the queue.
Queue: 20 30
```

A few concepts

💡 **Explain:** `Node(int val) : data(val), next(nullptr) {}`

- `Node(int val)` is the constructor declaration. It specifies that the constructor takes an integer parameter `val`.
- `:` introduces the member initializer list. The member initializer list initializes the data members of the `Node` struct.
- `data(val)` initializes the `data` member of the `Node` struct with the value passed as the parameter `val`.
- `next(nullptr)` initializes the `next` member of the `Node` struct with a null pointer (`nullptr`), indicating that initially, there is no next node.

So, when a `Node` object is created using this constructor, it sets the `data` member to the provided value `val` and sets the `next` pointer to `nullptr`, effectively creating a node with the specified data value and no connection to any other node (initializing it as the last node in a linked list). This constructor is particularly useful for initializing new nodes when building linked data structures such as queues, stacks, or linked lists.

💡 **Why are the `front` and `rear` pointers declared as private?**

The `front` and `rear` pointers in the `LinkedQueue` class are declared as private to **encapsulate the internal implementation details of the queue**. By keeping them private, the class hides these pointers **from external access**, allowing users to interact with the queue through well-defined public member functions **without needing to understand or manipulate the internal structure of the queue**. This approach enhances **data integrity**, as the class can control how these pointers are accessed and modified, preventing unintended changes. Additionally, encapsulation provides abstraction, separating the interface of the class from its implementation details, which promotes code flexibility and maintenance. Overall, making `front` and `rear` private follows best practices in object-oriented design, facilitating modular and maintainable code.

Circular queue

Coming soon ...